

I. 조건문 · 반복문 (문항 1~7)

문항 1. 두 수 중 더 큰 값 구하기

두 숫자를 입력받아 더 큰 값을 반환하는 함수를 작성하시오.

```
def bigger(a, b):  
    return a if a > b else b  
  
print(bigger(15, 22)) # 22
```

해설: 조건 표현식(삼항 연산자) "a if 조건 else b"를 사용하면 if-else문을 한 줄로 줄일 수 있다.

문항 2. 섭씨를 화씨로 변환하기

섭씨 온도를 입력받아 화씨 온도로 변환하는 함수를 작성하시오. (공식: 화씨 = 섭씨 × 9/5 + 32)

```
def c_to_f(c):  
    return c * 9/5 + 32  
  
print(c_to_f(25)) # 77.0
```

해설: 주어진 공식을 그대로 파이썬 수식으로 옮기면 된다. 곱셈과 나눗셈이 덧셈보다 먼저 계산되므로 괄호 없이도 올바르게 동작한다.

문항 3. 5부터 1까지 카운트다운 출력하기

while문을 이용하여 5부터 1까지 숫자를 거꾸로 출력하시오.

```
n = 5  
while n >= 1:  
    print(n, end=" ")  
    n -= 1
```

해설: while문은 조건이 참인 동안 반복 실행된다. n -= 1로 값을 1씩 줄여가며 5, 4, 3, 2, 1 순서로 출력한다.

문항 4. 문자열에서 특정 문자 개수 세기

문자열 "banana"에서 문자 'a'가 몇 번 나오는지 세는 코드를 작성하시오.

```
s = "banana"  
print(s.count("a")) # 3
```

해설: 문자열의 count() 메서드는 괄호 안의 문자(또는 부분 문자열)가 몇 번 등장하는지 세어준다.

문항 5. 리스트에 특정 값이 포함되어 있는지 확인하기

리스트 [3, 7, 9, 12]에 숫자 9가 포함되어 있는지 확인하는 코드를 작성하시오.

```
numbers = [3, 7, 9, 12]  
print(9 in numbers) # True
```

해설: in 연산자는 어떤 값이 리스트(또는 문자열) 안에 포함되어 있는지를 True/False로 알려준다.

문항 6. 리스트를 오름차순으로 정렬하기

리스트 [5, 2, 8, 1, 9]를 작은 수부터 큰 수 순서로 정렬하시오.

```
nums = [5, 2, 8, 1, 9]  
print(sorted(nums)) # [1, 2, 5, 8, 9]
```

해설: sorted() 함수는 원본 리스트는 그대로 두고 정렬된 새 리스트를 반환한다. 원본 자체를 바꾸고 싶다면 nums.sort()를 사용한다.

문항 7. 1부터 n까지 짝수만 출력하기

while문을 이용하여 1부터 10까지의 숫자 중 짝수만 출력하시오.

```
i = 1  
while i <= 10:  
    if i % 2 == 0:  
        print(i, end=" ")  
    i += 1
```

해설: 1부터 10까지 하나씩 확인하면서, 2로 나눈 나머지가 0인 수(짝수)만 골라서 출력한다.

II. 리스트 · 딕셔너리 · 문자열 (문항 8~14)

문항 8. 리스트에서 중복된 값 제거하기

리스트 [1, 2, 2, 3, 4, 4, 5]에서 중복된 값을 제거한 결과를 구하시오.

```
nums = [1, 2, 2, 3, 4, 4, 5]
print(list(set(nums))) # [1, 2, 3, 4, 5]
```

해설: set()은 중복을 허용하지 않는 자료형이므로, 리스트를 set으로 바꾸면 중복이 자동으로 제거된다. 다시 list()로 감싸 리스트 형태로 되돌린다.

문항 9. 두 리스트를 하나로 합치기

리스트 [1,2,3]과 [4,5,6]을 하나의 리스트로 합치는 코드를 작성하시오.

```
a = [1, 2, 3]
b = [4, 5, 6]
print(a + b) # [1, 2, 3, 4, 5, 6]
```

해설: 리스트끼리 + 연산자를 사용하면 두 리스트가 순서대로 이어진 새로운 리스트가 만들어진다.

문항 10. 딕셔너리에서 이름으로 점수 찾기

딕셔너리 {"민수":90, "지은":85, "현우":78}에서 "지은"의 점수를 출력하는 코드를 작성하시오.

```
scores = {"민수": 90, "지은": 85, "현우": 78}
print(scores["지은"]) # 85
```

해설: 딕셔너리는 {키: 값} 형태로 데이터를 저장하며, 대괄호 안에 키를 넣으면 해당하는 값을 바로 꺼낼 수 있다.

문항 11. 문장의 각 단어 첫 글자를 대문자로 바꾸기

문자열 "hello world"의 각 단어 첫 글자를 대문자로 바꾸는 코드를 작성하시오.

```
s = "hello world"
print(s.title()) # Hello World
```

해설: title() 메서드는 문자열 안의 모든 단어의 첫 글자를 대문자로, 나머지는 소문자로 자동 변환해준다.

문항 12. 회문(팰린드롬) 판별하기

앞으로 읽으나 뒤로 읽으나 같은 단어(회문)인지 판별하는 함수를 작성하시오.

```
def is_palindrome(s):
    return s == s[::-1]

print(is_palindrome("level")) # True
print(is_palindrome("hello")) # False
```

해설: s[::-1]은 문자열을 거꾸로 뒤집는 슬라이싱이다. 원래 문자열과 뒤집은 문자열이 같으면 회문이다.

문항 13. 자연수의 각 자리 숫자 합 구하기

자연수 4827의 각 자리 숫자(4, 8, 2, 7)를 모두 더한 값을 구하는 코드를 작성하시오.

```
n = 4827
total = 0
for ch in str(n):
    total += int(ch)
print(total) # 21
```

해설: 숫자를 str()로 문자열로 바꾸면 한 글자씩(자리수별로) 반복할 수 있다. 각 글자를 다시 int()로 숫자로 바꿔 더한다.

문항 14. 리스트를 리스트 컴프리헨션으로 만들기

리스트 컴프리헨션을 이용하여 1부터 10까지 숫자의 제곱으로 이루어진 리스트를 만드시오.

```
squares = [x**2 for x in range(1, 11)]
print(squares)
# [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

해설: 리스트 컴프리헨션 [식 for 변수 in 범위]는 반복문을 한 줄로 줄여서 새 리스트를 만드는 파이썬만의 간결한 문법이다.

III. 함수 · 응용 (문항 15~20)

문항 15. 재귀 함수로 리스트 합계 구하기

재귀 함수를 이용하여 리스트 [1,2,3,4,5]의 모든 원소의 합을 구하시오.

```
def rec_sum(lst):
    if not lst:
        return 0
    return lst[0] + rec_sum(lst[1:])

print(rec_sum([1, 2, 3, 4, 5])) # 15
```

해설: 재귀 함수는 자기 자신을 다시 호출하는 함수이다. 리스트가 비었을 때(기저 조건) 0을 반환하고, 그 전에는 첫 원소와 나머지 리스트의 합을 더한다.

문항 16. 두 수의 최대공약수(GCD) 구하기

유클리드 호제법을 이용하여 두 수 48과 18의 최대공약수를 구하는 함수를 작성하시오.

```
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

print(gcd(48, 18)) # 6
```

해설: 유클리드 호제법은 "두 수의 최대공약수는 큰 수를 작은 수로 나눈 나머지와 작은 수의 최대공약수와 같다"는 원리를 반복 적용한 것이다.

문항 17. 문장을 단어 단위로 거꾸로 배치하기

문장 "나는 학교에 간다"를 단어 순서를 거꾸로 하여 "간다 학교에 나는"으로 만드는 코드를 작성하시오.

```
def reverse_words(s):
    return " ".join(s.split()[::-1])

print(reverse_words("나는 학교에 간다"))
# 간다 학교에 나는
```

해설: split()으로 문장을 단어 리스트로 나누고, [::-1]로 순서를 뒤집은 뒤 join()으로 다시 하나의 문장으로 합친다.

문항 18. 튜플에서 값 꺼내기(언패킹)

좌표를 나타내는 튜플 (3, 7)에서 x와 y 값을 각각 변수에 저장하는 코드를 작성하시오.

```
point = (3, 7)
x, y = point
print(x, y) # 3 7
```

해설: 튜플 언패킹을 이용하면 튜플 안의 값들을 순서대로 여러 변수에 한 번에 나누어 담을 수 있다.

문항 19. 리스트에서 최댓값의 위치(인덱스) 찾기

리스트 [4, 9, 2, 15, 6]에서 가장 큰 값이 몇 번째(인덱스)에 있는지 찾는 코드를 작성하시오.

```
nums = [4, 9, 2, 15, 6]
print(nums.index(max(nums))) # 3
```

해설: max()로 리스트의 최댓값을 구하고, index()로 그 값이 리스트에서 몇 번째(0부터 시작)에 있는지 찾는다.

문항 20. 간단한 은행 잔액 관리 클래스 만들기

초기 잔액을 가지고, 입금(deposit)과 출금(withdraw)이 가능한 은행 계좌 클래스를 작성하시오. 잔액보다 많이 출금하려 하면 "잔액 부족"을 반환한다.

```
class Bank:
    def __init__(self, balance=0):
        self.balance = balance
    def deposit(self, amt):
        self.balance += amt
        return self.balance
    def withdraw(self, amt):
        if amt > self.balance:
            return "잔액 부족"
        self.balance -= amt
        return self.balance

b = Bank(10000)
b.deposit(5000)
print(b.withdraw(3000)) # 12000
```

해설: 클래스는 데이터(잔액)와 그 데이터를 다루는 함수(입금, 출금)를 하나로 묶은 것이다. __init__은 계좌를 처음 만들 때 초기 잔액을 설정하는 생성자이다.

